

EXACT 2026 - Submission Guide

The 2nd International XAI Challenge for Transparent Educational Question-Answering IEEE IJCNN 2026 Competition

This document explains exactly how to package and submit your system for evaluation. Please read it carefully, as submissions that do not follow this format may fail automated evaluation.

1. Overview

Your system is evaluated as a **live HTTP API endpoint**. During your grading slot, our evaluation server sends queries to your endpoint and collects the responses. Your endpoint must:

1. Be publicly reachable over the internet
2. Accept queries in the format described below
3. Return responses in the required JSON schema
4. Respond within **60 seconds per request**
5. Stay online for the **entire duration of your registered grading slot**

Evaluation rules:

- **One call per query:** each query is sent to your endpoint **exactly once**. There are no retries, so make sure your endpoint is stable. If a call fails or times out, that query is scored as incorrect.
- **60-second timeout:** any response taking longer than 60 seconds is treated as a failed answer for that query.
- **Sequential, one team at a time:** queries are sent **sequentially** (one after another, not in parallel), and each team is evaluated in its own slot. This keeps load predictable and ensures fairness, since the speed bonus is measured under the same conditions for everyone.

In addition, your underlying LLM must be served via **vLLM** (or a compatible OpenAI-style serving framework) so the committee can verify the model. See Section 6.

Test Composition

The test set contains **50 questions: 25 Type 1** (Logic-Based Educational Queries) and **25 Type 2** (Physics Problems). All 50 are sent to your single `/predict` endpoint during your grading slot. After grading, the committee will return to you the answers your model produced for each query, so you can review them.

2. Required Endpoints

Your deployment exposes **one prediction endpoint that you build**, plus the **model info endpoint** that vLLM provides automatically.

2.1 Prediction Endpoint (the one you build)

This is your **single** solution endpoint. It receives every competition query and returns the answer with explanation.

```
POST https://<your-host>/predict
Content-Type: application/json
```

It is **one endpoint that handles both Type 1 and Type 2**. You do not create separate endpoints per type. When a request arrives, you read the `type` field and route it internally to the correct pipeline:

```
POST /predict
|
+-- if type == "type1" -> logic pipeline    (FOL + solver)
+-- if type == "type2" -> physics pipeline (compute + CoT)
```

The endpoint path can be anything you like (`/predict`, `/`, `/api/solve`, etc.); just give us the exact URL. Internally it may call your vLLM server, symbolic solvers, retrieval modules, code execution, and so on. What matters is that this one URL accepts the competition input format and returns the competition output format.

2.2 Model Info Endpoint (provided by vLLM, not built by you)

You do **not** code this endpoint. When you launch your model with `vllm serve`, vLLM automatically exposes a standard endpoint that lists the loaded model:

```
GET https://<your-vllm-host>/v1/models
```

You only need to make sure this is reachable by the committee so we can verify which model is running. It is completely separate from your solution logic. If you run more than one model (allowed as long as the combined size loaded and running at any moment stays within 8B, per Q3 in the Official Q&A), each vLLM server has its own `/v1/models` URL.

You list all of your URLs (the prediction URL and every `/v1/models` URL) in a single text file inside your submission package (see Section 8). The committee may query any of them during your grading slot.

3. Input Format

The evaluation server sends a **POST** request to your `/predict` endpoint. The request body is a single JSON object. We use **one unified schema** for both query types: every field is always present, and fields that do not apply are left empty (`""` or `[]`). This way, you only need to check whether a field is empty rather than whether it exists.

3.1 Unified Input Schema

```
{
  "query_id": "T1_0001",
  "type": "type1",
  "query": "Is Student A eligible for graduation?",
  "premises": [
```

```

    "A student who has completed at least 120 credits is eligible for
    graduation.",
    "Student A has completed 118 credits."
  ],
  "options": ["Yes", "No", "Uncertain"]
}

```

3.2 Field Definitions

Field	Type	Description	Empty when
query_id	string	Unique query identifier	never
type	string	"type1" or "type2"	never
query	string	The question (Type 1) or the full problem statement (Type 2)	never
premises	list	Natural-language premises, 0-indexed (the first premise is index 0). Reference these indices in <code>premises_used</code>	[] for Type 2
options	list	The choice set for a multiple-choice question	[] for number/text questions and for Type 2

How to read options:

- If `options` is **non-empty**, it is a choice question. Your `answer` must be exactly one of the listed options.
- If `options` is **empty** (`[]`), the answer is free-form (a number or a short text). Return the value directly in `answer`.

3.3 Examples

Type 1, choice question (`options` non-empty):

```

{
  "query_id": "T1_0001",
  "type": "type1",
  "query": "Is Student A eligible for graduation?",
  "premises": ["A student with >= 120 credits is eligible.", "Student A has 118
  credits."],
  "options": ["Yes", "No", "Uncertain"]
}

```

Type 1, number/text question (`options` empty):

```

{
  "query_id": "T1_0002",
  "type": "type1",

```

```

    "query": "How many more credits does Student A need to graduate?",
    "premises": ["A student with >= 120 credits is eligible.", "Student A has 118 credits."],
    "options": []
  }
}

```

Type 2, physics problem (premises and options both empty):

```

{
  "query_id": "T2_0001",
  "type": "type2",
  "query": "Two resistors R1 = 4 ohm and R2 = 6 ohm are in parallel across a 12V battery. Find the total current.",
  "premises": [],
  "options": []
}

```

Important: Your single `/predict` endpoint must handle **both** query types. Use the `type` field to route the request to the correct internal pipeline. (See Q17 and Q18 in the Official Q&A.)

4. Output Format

Your endpoint must return a **JSON list** containing one result object per query. Even for a single query, return a list with one element. As with the input, we use **one unified schema**: every field is always present, and fields that do not apply are left empty (`""` or `[]`).

4.1 Unified Output Schema

```

[
  {
    "query_id": "T1_0001",
    "answer": "No",
    "unit": "",
    "explanation": "The eligibility rule requires at least 120 credits, but Student A has only 118, which is below the threshold. Therefore Student A is not eligible.",
    "premises_used": [0, 1],
    "reasoning": {
      "type": "fol",
      "steps": [
        "Rule: Credits(x) >= 120 => Eligible(x)",
        "Fact: Credits(StudentA) = 118",
        "118 < 120, premise not satisfied",
        "Conclusion: not Eligible(StudentA)"
      ]
    }
  }
]

```

```
}
]
```

In the input above, `premises[0]` is the eligibility rule and `premises[1]` is the fact about Student A's credits. Since both were used to reach the answer, `premises_used` is `[0, 1]`.

4.2 Field Definitions

Field	Type	Description	Empty when
query_id	string	Must match the <code>query_id</code> from the input	never
answer	string	The final answer. Type 1: the chosen option, number, or text. Type 2: the numerical value only (unit goes in unit)	never
unit	string	Unit of the answer, in ASCII (e.g., A, V, ohm, V/m, J, W, uF, nC)	"" for Type 1
explanation	string	Natural-language explanation. Must not be empty or null, even though it is not scored in this round	never
premises_used	list of int	The 0-based indices of the premises (from the input <code>premises</code> array) that you actually used to derive the answer	[] for Type 2
reasoning	object	Structured reasoning evidence (optional). Use <code>null</code> if not provided	<code>null</code> if not used

4.3 Examples

Type 1 response (`unit` empty, `premises_used` are 0-based indices):

```
[
  {
    "query_id": "T1_0001",
    "answer": "No",
    "unit": "",
    "explanation": "Student A has 118 credits, below the 120 required, so not eligible.",
    "premises_used": [0, 1],
    "reasoning": { "type": "fol", "steps": ["118 < 120", "not Eligible(StudentA)"] }
  }
]
```

Type 2 response (`unit` filled, `premises_used` empty):

```
[
```

```

{
  "query_id": "T2_0001",
  "answer": "5",
  "unit": "A",
  "explanation": "Two resistors in parallel give 2.4 ohm; 12V / 2.4 ohm = 5 A.",
  "premises_used": [],
  "reasoning": {
    "type": "cot",
    "steps": [
      "1/Req = 1/4 + 1/6 = 5/12",
      "Req = 2.4 ohm",
      "I = 12 / 2.4 = 5 A"
    ]
  }
}

```

4.4 The **reasoning** Object (Optional)

```

"reasoning": {
  "type": "fol",
  "steps": ["...", "..."]
}

```

- **type**: one that fits your system, e.g. "fol" (First-Order Logic), "cot" (Chain-of-Thought), or "proof" (structured proof)
- **steps**: a list of reasoning steps

This field is optional in this round (use `null` if you do not provide it) but recommended, as it will matter for reasoning-depth assessment (P3) in later stages.

4.5 Scoring for This Round

Base score:

Type	Criteria	Weight
Type 1	P1: answer correctness	50%
	P2: correctness of <code>premises_used</code>	50%
Type 2	P1: answer and unit both correct	100%

For Type 2, both the numerical `answer` and the `unit` must be correct to receive credit. Use standard ASCII units to ensure correct matching.

The `explanation` field is required but **not scored** in this round. It must still be present and non-empty.

Bonus points (added on top of the base score):

Bonus	Max	Condition
Dataset correction	+10%	Verified reports of dataset issues (see Q22 in the Official Q&A)
Inference speed	+10%	Awarded based on average inference time, computed only over correctly answered queries

Note on the speed bonus: Speed is measured only on queries you answer **correctly**. Returning fast but incorrect (or empty) responses earns **no** speed bonus. This prevents gaming the metric by sacrificing accuracy for speed. Among teams with correct answers, faster average inference time earns a higher share of the 10% bonus.

5. Notation Mapping (Unit & Symbol Fairness)

To ensure fairness in how mathematical and unit notation is matched, the committee will provide a **Notation Mapping CSV** before the evaluation phase.

5.1 How It Works

The committee provides a separate **Notation Mapping CSV** file with three columns:

Column	Description
<code>canonical_latex</code>	The canonical LaTeX form as it appears in our dataset
<code>meaning</code>	A plain description of the symbol
<code>your_notation</code>	You fill this in with the exact notation your model expects (leave blank to use the canonical form)

Before sending queries to your endpoint, the committee will **regex-replace** the notation in our problems with your declared notation. This way, your model receives input in the exact symbolic form it expects, and your answers are matched against ground truth in the same convention. You only need to fill in rows where your convention differs from the canonical form; blank rows are left as-is.

5.2 Symbols That May Appear in Problems

Below is the full set of symbols that may appear in the dataset. The Notation Mapping CSV will cover all of these.

Arithmetic and algebraic operators: `+`, `-`, `\times` (multiply), `\cdot` (dot multiply), `\div` (divide), `/` (fraction slash), `\pm` (plus-minus), `\mp` (minus-plus), `=`, `\approx`, `\neq`, `<`, `>`, `\leq`, `\geq`, `\propto` (proportional to), `\infty`

Powers, roots, fractions, and scientific notation: `a^b` (power / superscript), `a_b` (subscript), `\sqrt{}` (square root), `\sqrt[n]{}` (nth root), `\frac{a}{b}` (fraction), `\times 10^{n}` (scientific notation, e.g. `3 \times 10^{-6}`)

Greek letters (common in circuits and electrostatics): `\alpha`, `\beta`, `\gamma`, `\delta`, `\Delta` (change in), `\epsilon` / `\varepsilon` (permittivity), `\theta`, `\lambda` (linear charge density), `\mu` (micro prefix / permeability), `\pi`, `\rho` (resistivity / volume charge density), `\sigma` (conductivity / surface charge density), `\tau` (time constant), `\phi` / `\varphi` (potential / flux), `\Phi` (flux), `\omega` (angular frequency), `\Omega` (ohm)

Calculus and vector symbols (if any appear): `\int` (integral), `\sum` (sum), `\partial` (partial derivative), `\nabla` (nabla / del), `\vec{}` (vector), `\hat{}` (unit vector)

Units (electric circuits and electrostatics): `V` (volt), `A` (ampere), `\Omega` / `ohm` (ohm), `W` (watt), `J` (joule), `C` (coulomb), `F` (farad), `H` (henry), `T` (tesla), `Wb` (weber), `N` (newton), `V/m`, `N/C`, `Hz`, `s`, `m`, `eV`

Metric prefixes: `p` (pico), `n` (nano), `\mu` / `u` (micro), `m` (milli), `k` (kilo), `M` (mega), `G` (giga)

Other symbols: `\degree` / `°` (degree), `\%` (percent), `\angle` (angle)

For answer units: As stated in Section 4, the `unit` field in your Type 2 response must use **standard ASCII** (e.g., `V/m`, `ohm`, `uF`, `nC`). The Notation Mapping CSV ensures both the problem text and the unit matching follow a consistent convention.

6. Model Serving & Verification (vLLM)

To ensure fairness, all LLM components must be served via **vLLM** or a compatible OpenAI-style framework.

6.1 Why vLLM

Because an API endpoint is a black box, we need a standard way to confirm which model is actually running. vLLM exposes a standard `/v1/models` endpoint that reports the loaded model. This prevents teams from secretly routing to a larger model.

6.2 What You Must Do

Launch your model with vLLM, for example:

```
vllm serve meta-llama/Llama-3.1-8B-Instruct --port 8000
```

Make sure the `/v1/models` endpoint is reachable by the committee:

```
curl https://<your-vllm-host>/v1/models
```

This returns something like:

```
{
  "object": "list",
  "data": [
    {
      "id": "meta-llama/Llama-3.1-8B-Instruct",
      "object": "model",
      "owned_by": "vllm"
    }
  ]
}
```

The `id` must match the model you declared in your solution description. Third-party inference APIs (Together AI, Fireworks, Groq, Replicate, etc.) are **not allowed** because their model identity cannot be verified. See Q5 in the Official Q&A.

6.3 Using More Than One Model

The constraint is on total capacity: at any given moment during inference, the total parameters of all LLMs loaded and running together must stay within the 8B-class limit (per Q3 in the Official Q&A, where the prohibition on parallel inference refers to configurations that exceed 8B, such as two 8B models giving 16B). Multiple models that together stay within 8B are acceptable.

- **Allowed:** an 8B model for Type 1 and a separate 8B model for Type 2 used one at a time (sequential); or two 4B models running in parallel, since their combined size is 8B, which is within the limit.
- **Not allowed:** any configuration whose combined active size exceeds 8B at the same moment, such as two 8B models running in parallel (16B). This is what the Q&A means by parallel inference not being allowed.
- **Not allowed:** having LLMs whose combined size exceeds 8B resident and active on the GPU at the same time (e.g., two 8B models = 16B). The committee may inspect GPU memory usage during the Public Test Day to verify.

If you use more than one model:

- Declare **every** model in your solution description, with its parameter count
- Provide a `/v1/models` URL for **each** vLLM server
- For MoE models, count **total** parameters (see Q2)
- Non-LLM tools (solvers, retrieval, code execution) do not count toward the limit

7. Grading Slot Registration & Hosting Window

The submission deadline has been **extended to June 12, 2026**.

To keep evaluation organized, the committee will open a **slot registration form**. Each team registers **one grading slot**, and each slot lasts **one hour**.

What this means for you:

- You only need to keep your API endpoint(s) online **during your one-hour slot**, not for the entire phase.
- During your slot, the evaluation server will send all 50 test queries to your `/predict` endpoint and may inspect your `/v1/models` endpoint(s).
- Make sure your endpoint is stable and reachable for the full hour of your chosen slot.
- After grading, the committee will return the answers your model produced for each query.
- If you miss your slot, contact the committee on Discord (`#technical-support`) as soon as possible; rescheduling is subject to availability.

The slot registration form and available time windows will be announced on Discord and the official website.

8. Submission Package

Bundle everything into a **single ZIP file named after your team**: `<team_name>.zip`. The **submission form** and the **time-slot registration form** will be shared soon on Discord and the official website. Email to `ura.hcmut@gmail.com` is accepted as a backup. The submission deadline is **June 12, 2026**.

Your `<team_name>.zip` must contain:

1. **Solution description** (one page, PDF) that must include:
 - **Datasets used**: for each dataset, state its name, source/origin, the **number of samples** used, and a few **sample entries**. This covers the official EXACT dataset as well as any external, crawled, or synthetic data.
 - **Approach and method**: a clear overview of your pipeline (understanding, reasoning, explanation generation).
 - **Model size calculation**: list every LLM in your pipeline with its parameter count, and show that the total loaded and running at any single moment stays within the 8B-class limit (per Q3 in the Official Q&A). Models used one at a time, or smaller models in parallel whose combined size is within 8B, are both fine. For MoE models, state the **total** parameter count (not active), per Q2.
2. **Source code** (a `.zip` of your code)
3. **URLs text file** (`.txt`) containing your prediction URL and every `/v1/models` URL
4. **Notation Mapping CSV** filled in with your notation convention (see Section 5)

Example structure:

```
<team_name>.zip
├─ solution.pdf
├─ source_code.zip
├─ urls.txt
└─ notation_mapping.csv
```

9. Pre-Submission Checklist

Before you submit, confirm the following:

- `/predict` accepts both Type 1 and Type 2 queries (routed by `type`)
- `/predict` returns a JSON **list** of result objects
- Each result includes `query_id`, `answer`, `explanation`, and `premises_used`
- Choice questions (`options` non-empty): `answer` is exactly one of the options
- Type 1: `premises_used` contains the **0-based indices** of the premises actually used
- Type 2: includes a `unit` field in ASCII, and `premises_used` is `[]`

- explanation is present and non-empty (not null)
- query_id in the response matches the input
- Responses arrive within 60 seconds (each query is called only once, no retries)
- Endpoint is stable under sequential, one-at-a-time requests
- Every vLLM server exposes a reachable /v1/models reporting your declared model(s)
- All served models are open-source
- The total parameters of all LLMs loaded and running at any single moment stay within the 8B-class limit (models used one at a time, or parallel models combining to $\leq 8B$, are both fine)
- Registered a one-hour grading slot via the form
- Endpoint(s) stay online for the full hour of your slot
- Submission ZIP named <team_name>.zip containing: solution.pdf, source code zip, urls.txt, notation_mapping.csv
- Solution PDF lists each dataset with its source and number of samples used
- Solution PDF includes a total parameter calculation proving the solution is within 8B
- Notation Mapping CSV filled in with your notation convention
- Submitted before the June 12, 2026 deadline

10. A Simple Reference Flow

Evaluation Server

```

|
|  POST /predict { query_id, type, query, premises, options }
v
Your /predict endpoint
|
|-- route by type
|
|      +-- type1: NL + premises -> FOL -> solver -> answer + premises_used
(indices)
|
|      (if options non-empty, answer must be one of them)
|      +-- type2: parse -> compute (code/CoT) -> answer + unit
|
|  (internally calls your vLLM server(s) as needed)
v
Return [ { query_id, answer, unit, explanation, premises_used, reasoning } ]

```